

FMU
TECH
LEVEL UP



Prof. Hercules Ramos

DESDE 0

PRIMEIRO DIA.

Compiladores

abc **Aula Prática 01 — Construindo um Analisador Léxico com Lark**

Ambiente: Google Colab (principal) ou VS Code + Jupyter

Roteiro da Aula

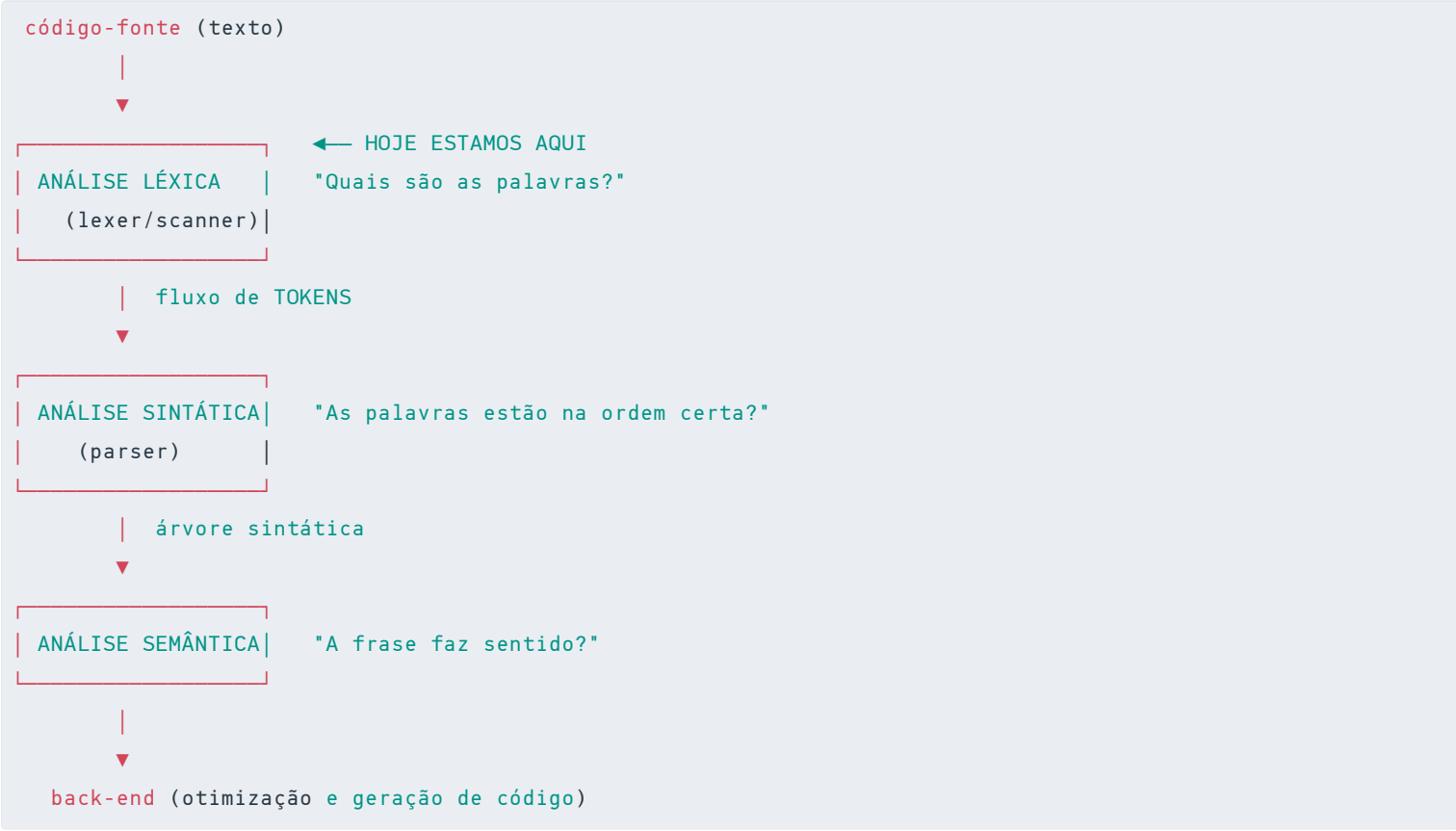
<https://whimsical.com/workspace84957/aula-cp-05-pratica-1-construindo-um-analisador-lexico-com-lark-T6Wotza8rJ1bBKDaktQS3X>

Bloco	Conteúdo
4	📁 Cenário B — PIX e extrato bancário (Exemplos B1 e B2)
5	Conexões com o mercado 2026
6	Exercícios (caderno + Colab)
7	🏆 Desafio para entrega

🎯 **Objetivo da aula:** ao final, você será capaz de **especificar tokens com expressões regulares, resolver conflitos de prioridade, reportar erros léxicos com linha e coluna e construir uma interface** que mostre o trabalho do lexer — tudo em Python, com a biblioteca **Lark**.

0. Recapitulação — Onde estamos no compilador?

Nas aulas anteriores vimos que um compilador é uma **linha de montagem**. O texto entra de um lado e, estação por estação, vai sendo transformado:



Na **Aula 03** usamos o módulo `re` do Python para "picotar" código em tokens. Funcionou, mas deu trabalho: tivemos que controlar a ordem dos padrões na mão, contar linhas e colunas na mão e tratar erros na mão.

Hoje vamos usar uma **ferramenta profissional**: o **Lark**, a mesma família de ferramentas de **Lex/Flex**, **ANTLR4** e **Tree-sitter**. Nós só **descrevemos** os tokens; o Lark **gera** o analisador.

💡 **Pergunta rápida para a turma:** se o compilador fosse um restaurante, o analisador léxico seria qual funcionário? (Guarde a resposta — ela aparece daqui a pouco.)

1. Teoria Relâmpago

1.1 A metáfora da cozinha 🍳

Imagine que você trabalha na cozinha de uma hamburgueria. Chega um papelzinho da impressora:

Antes de começar a cozinhar, o chapeiro faz, sem perceber, uma **análise léxica**:

- "PEDIDO" → *isso é uma palavra de comando*
- "2x" → *isso é uma quantidade*
- "X-Burger" → *isso é o nome de um prato*
- "R\$ 25,90" → *isso é um preço*
- "CUPOM" → *outra palavra de comando*
- "FRETE10" → *isso é um código*

Ele **não** está verificando se o pedido faz sentido (isso é sintaxe e semântica). Ele só está **separando e classificando as peças**. O analisador léxico é o funcionário que **separa os ingredientes em potinhos etiquetados** antes do cozinheiro (parser) montar o prato.

1.2 Os três conceitos que você não pode esquecer

Conceito	Definição	Na cozinha	No código
Token	A categoria lógica	a etiqueta do potinho: "PREÇO"	PRECO
Lexema	O texto real encontrado	o que está dentro do potinho: "R\$ 25,90"	'R\$ 25,90'
Padrão	A regra que descreve o token	"começa com R\$, tem número, vírgula e 2 centavos"	/R\\$? \d+, \d{2}/

Conforme a apostila (Unidade 5): *o token é a categoria lógica; o lexema é a sequência real de caracteres*. Além disso, o token pode carregar um **valor de atributo**: o lexema "R\$ 25,90" (texto) vira o **valor** 25.90 (número). Faremos isso no Exemplo A2.

1.3 O que o lexer JOGA FORA

Espaços, tabulações, quebras de linha e comentários servem para **humanos**. O lexer usa o espaço para **separar** palavras (sem ele, PEDID02x seria uma coisa só) e depois o **descarta**. No Lark isso é feito com a diretiva `%ignore`.

1.4 Três regras de ouro de todo lexer

Regra 1 — Maior casamento (maximal munch). O lexer tenta "morder" o maior pedaço possível. Diante de `FRETE10`, ele não para em `F` nem em `FRETE`: leva `FRETE10` inteiro.

Regra 2 — Prioridade. Quando dois padrões podem começar no mesmo lugar, vence quem tem **maior prioridade**. É como a fila do banco: idosos e gestantes (prioridade maior) passam na frente, mesmo tendo chegado junto.

Regra 3 — Erro léxico. Se nenhum padrão reconhece o caractere atual (por exemplo, um `@` perdido num pedido), o lexer **para e avisa**, informando **linha e coluna**. Um bom lexer ainda dá uma **dica** — como o VS Code faz quando sublinha algo em vermelho.

1.5 Como o Lark decide (versão simplificada)

Com `lexer="basic"`, o Lark funciona como um **scanner clássico** (igual ao Flex):

1. Ordena os tokens: **maior prioridade primeiro**; em empate, padrões mais "compridos/limitados" primeiro.
2. Na posição atual do texto, testa os padrões **nessa ordem** e fica com o **primeiro que casar**.
3. Se o lexema casou com um token de **regex** mas é igualzinho a uma **palavra reservada escrita entre aspas** (ex.: `"PEDIDO"`), o Lark **reclassifica** automaticamente como palavra reservada.
4. Avança o cursor, atualiza linha/coluna e repete.

⚠ Atenção: o passo 2 diz "o primeiro que casar", e não "o maior". Por isso a **prioridade** é tão importante no Lark — você vai ver isso quebrar ao vivo no Exemplo A1.

1.6 Mini-guia de expressões regulares usadas hoje

Regex	Significa	Casa com
<code>\d</code>	um dígito	<code>7</code>
<code>\d+</code>	um ou mais dígitos	<code>2026</code>
<code>\d{2}</code>	exatamente 2 dígitos	<code>09</code>
<code>\d{1,3}</code>	de 1 a 3 dígitos	<code>1</code> , <code>12</code> , <code>123</code>
<code>[A-Z]</code>	uma letra maiúscula	<code>F</code>
<code>[A-Z0-9]*</code>	zero ou mais letras/dígitos	<code>RETE10</code>
<code>"[^\"\\n]*"</code>	aspas, qualquer coisa exceto aspas/quebra, aspas	<code>"X-Burger"</code>
<code>R\\$</code>	o texto literal <code>R\$</code> (o <code>\$</code> precisa de <code>\</code>)	<code>R\$</code>
<code>?</code>	um espaço opcional	<code> </code> ou nada
<code>(\\.\\d{3})*</code>	grupos de milhar, zero ou mais vezes	<code>.250</code>
<code>\b</code>	fronteira de palavra (fim da palavra)	<code>—</code>
<code>/.../i</code>	ignora maiúsculas/minúsculas	<code>PIX</code> , <code>pix</code> , <code>Pix</code>

2. Preparando o Ambiente

2.1 Google Colab (recomendado)

1. Acesse **colab.research.google.com** e faça login com sua conta Google.
2. Clique em **Arquivo → Fazer upload de notebook** e escolha `Aula_Pratica_01_Analisador_Lexico.ipynb`.
3. Execute a **Célula 0** (instala o Lark). O `ipywidgets` já vem instalado no Colab.
4. Execute a **Célula 1** (ferramentas de visualização).
5. A partir daí, execute as células **em ordem** com `Shift + Enter`.

2.2 VS Code (alternativa)

1. Instale a extensão **Jupyter** (Microsoft).
2. No terminal: `pip install lark ipywidgets ipykernel`
3. Abra o `.ipynb`, escolha o interpretador Python no canto superior direito e execute as células.

2.3 Célula 0 — Instalação e importações

```
# =====
# CÉLULA 0 – PREPARAÇÃO DO AMBIENTE
# =====
# No Google Colab, o ipywidgets já vem instalado.
# O Lark precisa ser instalado (leva ~5 segundos).
# No VS Code, rode no terminal: pip install lark ipywidgets ipykernel
# -----
%pip install -q lark ipywidgets

import html
import lark
from lark import Lark
from lark.exceptions import UnexpectedCharacters, UnexpectedInput
```

```
import ipywidgets as widgets
from IPython.display import display, HTML

print(f"✅ Lark versão {lark.__version__} pronto!")
print(f"✅ ipywidgets versão {widgets.__version__} pronto!")
```

🔦 `UnexpectedCharacters` é a **exceção de erro léxico** do Lark. Ela traz `line`, `column` e `char` — tudo o que precisamos para um relatório de erro decente.

2.4 Célula 1 — Ferramentas de visualização

Estas funções **não fazem análise léxica**; elas apenas desenham o resultado. Três ideias merecem atenção:

- Cada token do Lark carrega `type` (o token), `value` (o lexema), `line`, `column`, `start_pos` e `end_pos`.
- `texto_colorido_html` usa `start_pos` / `end_pos` para pintar cada lexema **no lugar exato** do texto — é assim que funciona o *syntax highlighting* de editores como o VS Code (que usa TextMate/Tree-sitter por baixo).
- `erro_lexico_html` desenha a famosa "setinha" `^` embaixo do caractere com problema.

```
# =====
# CÉLULA 1 – FERRAMENTAS DE VISUALIZAÇÃO (usadas por todos os exemplos)
# =====
# Estas funções NÃO fazem análise léxica. Elas só "desenham"
# os tokens de forma bonita (tabela e texto colorido).

# Paleta de cores: cada categoria de token ganha uma cor.
CORES = {
    # --- Palavras reservadas (azul) ---
    "PEDIDO": "#1565c0", "ENTREGA": "#1565c0", "CUPOM": "#1565c0",
    "OBS": "#1565c0", "TAXA": "#1565c0", "PAGAMENTO": "#1565c0",
    "PIX": "#1565c0", "TED": "#1565c0", "BOLETO": "#1565c0",
    "TARIFA": "#1565c0", "SALDO_INICIAL": "#1565c0",
    "PARA": "#5e35b1", "DE": "#5e35b1", "EM": "#5e35b1",
    "DIRECAO": "#6a1b9a", "FORMA_PGTO": "#6a1b9a",
    # --- Literais (verde / laranja) ---
    "QTD": "#ef6c00", "NUMERO": "#ef6c00",
    "PRECO": "#2e7d32", "VALOR": "#2e7d32",
    "ITEM": "#c62828", "TEXTO": "#c62828", "DESCRICAO": "#c62828",
    "CODIGO": "#00838f",
    # --- Chaves PIX e datas (tons de marrom / cinza) ---
    "CHAVE_EMAIL": "#ad1457", "CHAVE_CPF": "#ad1457",
    "CHAVE_CNPJ": "#ad1457", "CHAVE_TELEFONE": "#ad1457",
    "CHAVE_ALEATORIA": "#ad1457",
    "DATA": "#4e342e", "HORA": "#4e342e",
}

def cor_do_token(tipo):
    """Devolve a cor da categoria (cinza se não estiver na paleta)."""
    return CORES.get(tipo, "#455a64")

def tabela_tokens_html(tokens, titulo="Tabela de Tokens"):
    """Monta uma tabela HTML com: n°, TOKEN (categoria), LEXEMA, linha e coluna."""
    linhas = ""
    for i, t in enumerate(tokens, start=1):
        cor = cor_do_token(t.type)
```

```

linhas += (
    f"<tr><td>{i}</td>"
    f"<td><b style='color:{cor}'>{t.type}</b></td>"
    f"<td><code>{html.escape(str(t.value))}</code></td>"
    f"<td>{t.line}</td><td>{t.column}</td></tr>"
)
return f"""
<h4 style='margin:4px 0'>{titulo} ({len(tokens)} tokens)</h4>
<table style='border-collapse:collapse;font-family:monospace;font-size:13px'>
  <tr style='background:#263238;color:white'>
    <th style='padding:4px 10px'>#</th><th style='padding:4px 10px'>TOKEN</th>
    <th style='padding:4px 10px'>LEXEMA</th><th style='padding:4px 10px'>LINHA</th>
    <th style='padding:4px 10px'>COLUNA</th>
  </tr>
  {linhas}
</table>"""

def texto_colorido_html(texto, tokens, transformar=None):
    """Pinta cada lexema no texto original, como faz o 'syntax highlight' do VS Code.
    Usa t.start_pos e t.end_pos: a posição exata de cada token no texto.
    'transformar' (opcional) troca o lexema exibido – usado para mascarar dados (LGPD)."""
    saida, cursor = "", 0
    for t in tokens:
        saida += html.escape(texto[cursor:t.start_pos]) # espaços/comentários
        cor = cor_do_token(t.type)
        saida += (f"<span title='{t.type}' style='color:{cor};font-weight:bold;"
                  f"border-bottom:2px solid {cor}'>"
                  f"{html.escape(transformar(t) if transformar else texto[t.start_pos:t.end_pos])}</span>")
        cursor = t.end_pos
    saida += html.escape(texto[cursor:])
    return ("<pre style='background:#fafafa;border:1px solid #ddd;padding:10px;"
            f"font-size:14px;line-height:1.7'>{saida}</pre>"
            "<small>💡 Passe o mouse sobre um lexema para ver o nome do token.</small>")

def erro_lexico_html(texto, erro):
    """Mostra um erro léxico com linha, coluna e uma 'setinha' apontando o problema."""
    linha_txt = texto.splitlines()[erro.line - 1] if texto.splitlines() else ""
    seta = " " * (erro.column - 1) + "^"
    return f"""
<div style='background:#ffebee;border-left:5px solid #c62828;padding:10px'>
  <b>✖ ERRO LÉXICO</b> na linha <b>{erro.line}</b>, coluna <b>{erro.column}</b>:
  caractere inesperado <code>{html.escape(repr(erro.char))}</code>
  <pre style='margin:6px 0'>{html.escape(linha_txt)}\n{seta}</pre>
  <small>{html.escape(getattr(erro, 'dica', ''))}</small>
</div>"""

print("✅ Ferramentas de visualização carregadas!")

```



CENÁRIO A — Delivery de Comida (estilo iFood)

Contexto de mercado

Aplicativos como **iFood**, **Rappi** e **Aiqfome** conversam com milhares de restaurantes. Muitos restaurantes usam sistemas de PDV antigos, que recebem pedidos em **formatos de texto** (arquivos, integrações, impressoras térmicas da cozinha). Alguém precisa **ler esse texto** e transformá-lo em dados. Esse "alguém" começa por um **analisador léxico**.

Vamos criar a nossa mini-linguagem de pedidos, a **"PedidoLang"**:

```
PEDIDO 2x "X-Burger" R$ 25,90 CUPOM FRETE10
```

Exemplo A1 — Comanda Digital (passo a passo)

Vamos construir o lexer **em camadas**, como se estivéssemos montando um hambúrguer.

Passo 1 — Liste os tokens no papel (antes de programar!)

Todo projeto de lexer começa com uma **tabela de tokens**. Olhe o pedido e pergunte: *quais "tipos de peça" existem aqui?*

Token	Descrição	Exemplos de lexema	Tipo
PEDIDO	comando de item	PEDIDO	palavra reservada
ENTREGA	endereço de entrega	ENTREGA	palavra reservada
CUPOM	aplicação de cupom	CUPOM	palavra reservada
QTD	quantidade	2x , 10x	literal
ITEM	nome do produto	"X-Burger"	literal (texto)
PRECO	valor em reais	R\$ 25,90	literal
CODIGO	código do cupom	FRETE10	identificador

Passo 2 — A regra inicial

```
start: _token*
_token: PEDIDO | ENTREGA | CUPOM | QTD | ITEM | PRECO | CODIGO
```

- `start` aceita **qualquer sequência** de tokens (`*` = zero ou mais).
- Não estamos verificando a **ordem** das peças: isso é trabalho do parser (próximas aulas).
- O `_` em `_token` pede ao Lark para não criar um nó extra na árvore (detalhe estético).

Passo 3 — Palavras reservadas

```
PEDIDO: "PEDIDO"
ENTREGA: "ENTREGA"
CUPOM: "CUPOM"
```

Texto **entre aspas** no Lark é um padrão **literal**: casa exatamente com aquela sequência de caracteres. No Lark, **nomes de tokens são sempre em MAIÚSCULAS** (regras sintáticas ficam em minúsculas).

Passo 4 — Literais com expressões regulares

```
QTD.2: /\d+x/
ITEM:  /"[^"\n]"*/
PRECO: /R\$ ?\d+, \d{2}/
```

- `QTD.2` → o `.2` é a **prioridade**. O padrão casa "um ou mais dígitos seguidos de x".
- `ITEM` → aspas, qualquer coisa que não seja aspas ou quebra de linha, aspas.

- `PRECO` → `R$`, espaço opcional, dígitos, vírgula e **exatamente** 2 centavos.

Passo 5 — Identificadores

```
CODIGO: /[A-Z][A-Z0-9]*/
```

Começa com letra maiúscula, seguida de letras ou dígitos. Repare: **"PEDIDO" também casa com esse padrão!** Quem resolve? O Lark: quando o lexema de um token regex é **idêntico** a uma palavra reservada literal, ele reclassifica. Por isso `PEDIDO` vira `PEDIDO`, mas `PEDIDOS` continua `CODIGO`.

Passo 6 — Ruído

```
%ignore /\s+|[\t\r\n]+/
```

Espaço, tabulação e quebras de linha são reconhecidos e **descartados**.

Passo 7 — Criando o lexer

```
lexer_a1 = Lark(grammar_a1, parser="lalr", lexer="basic")
tokens = list(lexer_a1.lex(texto))
```

- `lexer="basic"` → scanner clássico, independente do parser (o modo ideal para estudar o léxico isoladamente).
- `.lex(texto)` → devolve um **gerador** de tokens; o `list(...)` coleta todos.

Passo 8 — 🌟 O bug que ensina: por que `PRECO.2` ?

Na primeira versão deste exemplo, o `PRECO` estava **sem** prioridade. Resultado:

```
🌟 Erro na coluna 23: caractere '$'
Tokens lidos antes do erro: [Token('CODIGO', 'R')]
```

O que aconteceu? Na posição do `R$`, dois padrões podiam começar: `CODIGO` (que casa `R`) e `PRECO` (que casa `R$ 25,90`). Com a mesma prioridade, o Lark testou `CODIGO` primeiro, "comeu" o `R` e deixou o `$` órfão. **Nenhum token começa com `$`** → erro léxico.

A solução foi dar **prioridade 2** ao `PRECO`. Esse é o conflito mais clássico de lexers: **identificador × literal que começa com letra**. O notebook tem uma célula de **Experimento A1** que reproduz o erro ao vivo.

🧠 **Moral da história:** quando um token "rouba" o começo de outro, suba a prioridade do token mais específico.

Código completo — Exemplo A1

```
# =====
# EXEMPLO A1 – COMANDA DIGITAL (lexer de UMA linha de pedido)
# Cenário: o app do restaurante manda um pedido em texto para a cozinha.
# =====

gramatica_a1 = r"""
// ----- Regra inicial -----
// "start" aceita qualquer sequência de tokens.
// Aqui NÃO estamos validando a ordem (isso é papel do parser).
start: _token*
_token: PEDIDO | ENTREGA | CUPOM | QTD | ITEM | PRECO | CODIGO

// ----- 1) Palavras reservadas -----
PEDIDO: "PEDIDO"
ENTREGA: "ENTREGA"
```

```

CUPOM:      "CUPOM"

// ----- 2) Literais -----
QTD.2:   /\d+x/           // 2x, 10x (prioridade 2: vence NUMERO se existir)
ITEM:    /"[^"\n]*"/       // "X-Burger" (texto entre aspas)
PRECO.2: /R\$ ?\d+,\d{2}/   // R$ 25,90 (prioridade 2: senão o "R" vira CODIGO!)

// ----- 3) Identificadores -----
CODIGO:  /[A-Z][A-Z0-9]*/   // FRETE10, DESC15 (códigos de cupom)

// ----- 4) Ruído (descartado) -----
%ignore /\t\r\n\+/
"""

# lexer="basic" -> o Lark usa um scanner clássico (igual Lex/Flex):
# lê o texto da esquerda para a direita e devolve tokens, um por vez.
lexer_a1 = Lark(grammar_a1, parser="lalr", lexer="basic")

def tokenizar_a1(texto):
    """Devolve a lista de tokens de um pedido."""
    return list(lexer_a1.lex(texto))

# ---- Teste rápido no console ----
pedido = 'PEDIDO 2x "X-Burger" R$ 25,90 CUPOM FRETE10'
print(f"Entrada: {pedido}\n")
for t in tokenizar_a1(pedido):
    print(f"  {t.type:<8} -> {t.value!r}")

```

Saída esperada:

```

Entrada: PEDIDO 2x "X-Burger" R$ 25,90 CUPOM FRETE10

PEDIDO    -> 'PEDIDO'
QTD        -> '2x'
ITEM       -> '"X-Burger"'
PRECO      -> 'R$ 25,90'
CUPOM      -> 'CUPOM'
CODIGO     -> 'FRETE10'

```

Passo 9 — Interface com ipywidgets

O **ipywidgets** cria componentes gráficos (caixas de texto, botões, abas) dentro do notebook. A receita é sempre a mesma, em 4 ingredientes:

1. **Criar os widgets** — `Text`, `Dropdown`, `Button`, `Output`.
2. **Escrever uma função "callback"** — o que acontece quando o usuário clica.
3. **Ligar a função ao evento** — `botao.on_click(funcao)` ou `widget.observe(funcao, names="value")`.
4. **Exibir** — `display(...)`.

O widget `Output` funciona como uma "tela" onde tudo o que for exibido com `with saida:` aparece dentro dele.

```

# =====
# EXEMPLO A1 – INTERFACE (ipywidgets)
# =====

```

```

exemplos_a1 = {
    "Pedido simples": 'PEDIDO 2x "X-Burger" R$ 25,90',
    "Pedido com cupom": 'PEDIDO 1x "Pizza Calabresa" R$ 54,90 CUPOM FRETE10',
    "Pedido com entrega": 'PEDIDO 3x "Coxinha" R$ 7,50 ENTREGA "Rua Vergueiro, 1000"',
    "❌ Com erro (@)": 'PEDIDO 2x "Açaí 500ml" R$ 19,90 @CUPOM',
}

entrada_a1 = widgets.Text(value=exemplos_a1["Pedido simples"],
                           description="Pedido:", layout=widgets.Layout(width="95%"))
seletor_a1 = widgets.Dropdown(options=list(exemplos_a1), description="Exemplos:")
botao_a1 = widgets.Button(description="🔍 Tokenizar", button_style="primary")
saida_a1 = widgets.Output()

def ao_escolher_exemplo_a1(mudanca):
    entrada_a1.value = exemplos_a1[mudanca["new"]]

def ao_clicar_a1(_):
    saida_a1.clear_output()
    with saida_a1:
        texto = entrada_a1.value
        try:
            tokens = tokenizar_a1(texto)
            display(HTML(texto_colorido_html(texto, tokens)))
            display(HTML(tabela_tokens_html(tokens)))
        except UnexpectedCharacters as erro:
            display(HTML(erro_lexico_html(texto, erro)))

seletor_a1.observe(ao_escolher_exemplo_a1, names="value")
botao_a1.on_click(ao_clicar_a1)

display(widgets.HTML("<h3>🍔 Comanda Digital – Analisador Léxico</h3>"),
         seletor_a1, entrada_a1, botao_a1, saida_a1)
ao_clicar_a1(None) # já mostra o primeiro resultado

```

O que testar na interface:

- Escolha **"Pedido com entrega"** e observe o endereço reconhecido como `ITEM` (entre aspas).
- Escolha **"❌ Com erro (@)"**: o lexer aponta **linha 1, coluna 33** e desenha a setinha embaixo do `@`.
- Digite `PEDIDOS 2x` e veja `PEDIDOS` virar `CODIGO` (não é palavra reservada!).
- Digite `pedido 2x` (minúsculo) e veja o erro. Por quê? (Resposta: `"PEDIDO"` é literal sensível a maiúsculas, e `CODIGO` só aceita maiúsculas. O Exemplo A2 resolve isso.)

Exemplo A2 — Sistema de Pedidos do Restaurante (versão profissional)

O A1 lê **uma linha**. Um sistema real lê **um pedido inteiro**, escrito por pessoas diferentes, com letras maiúsculas e minúsculas, comentários e erros de digitação.

O que muda do A1 para o A2

Recurso	A1	A2
Linhas	uma	várias (linha/coluna importam!)

Recurso	A1	A2
Maiúsculas/minúsculas	só PEDIDO	PEDIDO , pedido , Pedido (/i)
Comentários	não	# Mesa 12 é descartado
Palavras reservadas	3	7 (inclui OBS , TAXA , PAGAMENTO , formas de pagamento)
Preço	R\$ 25,90	também R\$ 1.250,00 (milhar)
Erros	mensagem padrão	mensagem + dica amigável
Valor do token	só texto	texto → número (25.90)
Interface	1 tabela	4 abas (colorido, tokens, comanda, estatística)

Passo 1 — Palavras reservadas "à prova de maiúsculas"

```
PEDIDO.3: /pedido\b/i
```

Três detalhes, cada um com um motivo:

- `/.../i` — aceita `PEDIDO` , `pedido` , `Pedido` .
- `\b` — fronteira de palavra. Sem ele, `PEDIDOS` seria quebrado em `PEDIDO` + `S` . Com ele, `PEDIDOS` não casa com a palavra reservada e vira `CODIGO` .
- `.3` — prioridade maior que qualquer identificador. Como agora a palavra reservada é **regex** (e não literal entre aspas), a reclassificação automática do Lark não se aplica: quem decide é a prioridade.

Passo 2 — Uma palavra reservada com várias formas

```
FORMA_PGTO.3: /(pix|cart[aão]|dinheiro|vale_refeicao)\b/i
```

Um único token para o **conjunto** de formas de pagamento. O `[aão]` aceita `cartao` e `cartão` . (Exatamente como linguagens reais agrupam `true` / `false` em um token `BOOLEAN` .)

Passo 3 — Preço com separador de milhar

```
PRECO.2: /R\$ ?\d{1,3}(\.\d{3})*,\d{2}/
```

Leia em voz alta: *"R\$, espaço opcional, de 1 a 3 dígitos, depois zero ou mais grupos de ponto + 3 dígitos, vírgula, 2 dígitos"*. Casa com `R$ 9,90` , `R$ 25,90` e `R$ 1.250,00` .

Passo 4 — Comentários

```
COMENTARIO: /#[^\n]*/
%ignore COMENTARIO
```

Tudo depois de `#` até o fim da linha é descartado — igual ao Python.

Passo 5 — Erros com dicas (o "Stack Overflow embutido")

```
DICAS_A2 = {
    "'": 'Parece que você abriu aspas e não fechou...',
    "$": 'Preço sem o "R"? Escreva no formato R$ 25,90.',
    ".": 'Use vírgula nos centavos: R$ 25,90 (e não 25.90).',
}
```

A ideia é simples: olhamos o **caractere que causou o erro** (`erro.char`) e escolhemos uma dica. Compiladores modernos como **Rust** e **Elm** ficaram famosos justamente por mensagens de erro assim, que ensinam em vez de só reclamar.

Passo 6 — Do lexema ao valor

```
def valor_do_preco(lexema):  
    return float(lexema.replace("R$", "").strip().replace(".", "").replace(",", "."))
```

"R\$ 1.250,90" → tira o R\$ → "1.250,90" → tira o ponto → "1250,90" → troca vírgula por ponto → 1250.9 .Esse é o **valor do atributo** do token.

Código completo — Exemplo A2 (lexer)

```
# =====  
# EXEMPLO A2 – SISTEMA DE PEDIDOS DO RESTAURANTE (multilinha)  
# Novidades em relação ao A1:  
#   • palavras reservadas sem diferenciar maiúsculas/minúsculas  
#   • comentários (#) descartados  
#   • formas de pagamento, observações, taxa de entrega  
#   • preços com milhar (R$ 1.250,00)  
#   • mensagens de erro amigáveis (linha, coluna e dica)  
#   • conversão do LEXEMA em VALOR (texto "R$ 25,90" -> número 25.9)  
# =====  
  
gramatica_a2 = r"""  
start: _token*  
_token: PEDIDO | ENTREGA | CUPOM | OBS | TAXA | PAGAMENTO | FORMA_PGTO  
       | QTD | PRECO | TEXTO | CODIGO | NUMERO  
  
// ----- 1) Palavras reservadas (prioridade 3) -----  
// /i = ignora maiúsculas/minúsculas   \b = "fim de palavra"  
// Sem o \b, a palavra "PEDIDOS" viraria PEDIDO + "S"!  
PEDIDO.3:    /pedido\b/i  
ENTREGA.3:   /entrega\b/i  
CUPOM.3:     /cupom\b/i  
OBS.3:       /obs\b/i  
TAXA.3:      /taxa\b/i  
PAGAMENTO.3: /pagamento\b/i  
FORMA_PGTO.3: /(pix|cart[aã]o|dinheiro|vale_refeicao)\b/i  
  
// ----- 2) Literais (prioridade 2) -----  
QTD.2:    /\d+x\b/i           // 2x  3X  10x  
PRECO.2:  /R\$ ?\d{1,3}(\.\d{3})*,\d{2}/ // R$ 9,90  R$ 1.250,00  
TEXTO:    /"[^"\n]*/           // "X-Burger", "sem cebola"  
NUMERO:   /\d+/                // números soltos (ex.: mesa 12)  
  
// ----- 3) Identificadores -----  
CODIGO:   /[A-Za-z_][A-Za-z0-9_]*/ // FRETE10, desc15  
  
// ----- 4) Ruído -----  
COMENTARIO: /#[^\n]*/  
%ignore COMENTARIO  
%ignore /[ \t\r\n]+/
```

```

lexer_a2 = Lark(grammar_a2, parser="lalr", lexer="basic")

# Dicas amigáveis para os erros mais comuns (o "Stack Overflow" do nosso lexer)
DICAS_A2 = {
    "'": 'Parece que você abriu aspas e não fechou. Todo texto precisa de "abre" e "fecha".',
    "$": 'Preço sem o "R"? Escreva no formato R$ 25,90.',
    ".": 'Use vírgula nos centavos: R$ 25,90 (e não 25.90).',
}

def tokenizar_a2(texto):
    """Tokeniza o pedido. Se houver erro léxico, anexa uma dica ao erro."""
    try:
        return list(lexer_a2.lex(texto))
    except UnexpectedCharacters as erro:
        erro.dica = DICAS_A2.get(erro.char, "Caractere não pertence ao alfabeto da linguagem de pedidos.")
        raise

def valor_do_preco(lexema):
    """Converte o LEXEMA 'R$ 1.250,90' no VALOR 1250.90 (atributo do token)."""
    return float(lexema.replace("R$", "").strip().replace(".", "").replace(",", "."))

# Teste no console
pedido_a2 = '''# Mesa 12 – Hamburgueria do Zé
PEDIDO 2x "X-Burger" R$ 25,90
pedido 1x "Batata Frita G" R$ 18,50 OBS "sem sal"
TAXA R$ 7,99
CUPOM desc10
PAGAMENTO pix'''

for t in tokenizar_a2(pedido_a2):
    print(f"L{t.line:<2} C{t.column:<3} {t.type:<10} {t.value!r}")

```

Saída esperada (repare nas colunas L = linha, C = coluna):

L2	C1	PEDIDO	'PEDIDO'
L2	C8	QTD	'2x'
L2	C11	TEXTO	'"X-Burger"'
L2	C22	PRECO	'R\$ 25,90'
L3	C1	PEDIDO	'pedido'
L3	C8	QTD	'1x'
L3	C11	TEXTO	'"Batata Frita G"'
L3	C28	PRECO	'R\$ 18,50'
L3	C37	OBS	'OBS'
L3	C41	TEXTO	'"sem sal"'
L4	C1	TAXA	'TAXA'
L4	C6	PRECO	'R\$ 7,99'
L5	C1	CUPOM	'CUPOM'
L5	C7	CODIGO	'desc10'
L6	C1	PAGAMENTO	'PAGAMENTO'
L6	C11	FORMA_PGTO	'pix'

Note que a linha 1 (o comentário) **não gerou nenhum token**, mas continua sendo contada: o primeiro token está na linha 2.

Passo 7 — A comanda: uma espiada na próxima fase 🕵️

A aba  **Comanda** transforma a lista de tokens em um pedido de verdade (itens, subtotal, desconto, taxa, total). Para isso, a função `montar_comanda` procura **padrões na sequência de tokens**, como `QTD TEXTO PRECO`.

🔍 **Perceba:** isso **já não é mais análise léxica!** Verificar que depois de `QTD` vem `TEXTO` e depois `PRECO` é **análise sintática** (feita à mão). E recusar o cupom `GANHEI100` porque ele não existe é **análise semântica** (regra de negócio). Nas próximas aulas o Lark fará a parte sintática para nós, com uma gramática de verdade.

Código completo — Exemplo A2 (interface)

```
# =====
# EXEMPLO A2 - INTERFACE COMPLETA (abas: colorido | tokens | comanda | estatística)
# =====

from collections import Counter

CUPONS = {"DESC10": 0.10, "DESC15": 0.15, "PRIMEIRACOMPRA": 0.20} # % de desconto
CUPONS_FRETE = {"FRETEGRATIS", "FRETE10"} # zeram a taxa

def montar_comanda(tokens):
    """ESPIADA NA PRÓXIMA FASE: usa a sequência de tokens para montar a comanda.
    Isto já é um 'mini-parser' feito à mão: olhamos padrões como QTD TEXTO PRECO."""
    itens, taxa, cupom, pagamento, endereco, obs = [], 0.0, None, "não informado", None, []
    i = 0
    while i < len(tokens):
        t = tokens[i]
        prox = tokens[i + 1] if i + 1 < len(tokens) else None
        if t.type == "QTD" and i + 2 < len(tokens) and tokens[i+1].type == "TEXTO" and tokens[i+2].type == "PRECO":
            qtd = int(t.value[:-1])
            nome = tokens[i+1].value.strip(' ')
            preco = valor_do_preco(tokens[i+2].value)
            itens.append((qtd, nome, preco))
            i += 3
            continue
        if t.type == "TAXA" and prox and prox.type == "PRECO":
            taxa = valor_do_preco(prox.value)
        elif t.type == "CUPOM" and prox and prox.type == "CODIGO":
            cupom = prox.value.upper()
        elif t.type == "PAGAMENTO" and prox and prox.type == "FORMA_PGTO":
            pagamento = prox.value.upper()
        elif t.type == "ENTREGA" and prox and prox.type == "TEXTO":
            endereco = prox.value.strip(' ')
        elif t.type == "OBS" and prox and prox.type == "TEXTO":
            obs.append(prox.value.strip(' '))
        i += 1

    subtotal = sum(q * p for q, _, p in itens)
    desconto = subtotal * CUPONS.get(cupom, 0) if cupom else 0
    if cupom in CUPONS_FRETE:
        taxa = 0.0
    total = subtotal - desconto + taxa

    brl = lambda v: f"R$ {v:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")
```

```

linhas = "".join(f"<tr><td>{q}x</td><td>{html.escape(n)}</td><td align='right'>{brl(p)}</td>"
                f"<td align='right'>{brl(q*p)}</td></tr>" for q, n, p in itens)

aviso_cupom = ""
if cupom and cupom not in CUPONS and cupom not in CUPONS_FRETE:
    aviso_cupom = f"<p style='color:#c62828'>⚠ Cupom {cupom} não existe (o LEXER aceitou, mas a REGRA DE
NEGÓCIO recusou!)</p>"
    return f"""
<div style='font-family:monospace;max-width:520px;border:1px dashed #999;padding:12px'>
    <h3 style='margin:0'>🍷 COMANDA DA COZINHA</h3>
    <table style='width:100%'>{linhas}</table><hr>
    Subtotal: <b>{brl(subtotal)}</b><br>
    Desconto ({cupom or '-'}): <b>- {brl(desconto)}</b><br>
    Taxa de entrega: <b>{brl(taxa)}</b><br>
    <h3>TOTAL: {brl(total)}</h3>
    Pagamento: <b>{pagamento}</b><br>
    Entrega: <b>{html.escape(endereco) if endereco else 'retirada no balcão'}</b><br>
    Observações: {html.escape('; '.join(obs)) or '-'}
    {aviso_cupom}
</div>"""

def estatisticas_html(tokens):
    contagem = Counter(t.type for t in tokens)
    barras = "".join(
        f"<div style='margin:2px 0'><code style='display:inline-block;width:120px'>{tipo}</code>"
        f"<span style='display:inline-block;background:{cor_do_token(tipo)};height:14px;width:{n*25}px'></spa
n> {n}</div>"
        for tipo, n in contagem.most_common())
    return f"<h4>Frequência de cada categoria de token</h4>{barras}"

exemplos_a2 = {
    "Pedido completo (delivery)": '''# Pedido #4821 – Hamburgueria do Zé
PEDIDO 2x "X-Burger" R$ 25,90
PEDIDO 1x "Batata Frita G" R$ 18,50 OBS "sem sal"
PEDIDO 3x "Refrigerante Lata" R$ 6,00
ENTREGA "Rua das Flores, 123 - Apto 45"
TAXA R$ 7,99
CUPOM DESC10
PAGAMENTO pix''',
    "Pedido grande (evento)": '''# Festa da empresa – Pizzaria Bella
pedido 10x "Pizza Grande Mussarela" R$ 59,90
pedido 5x "Pizza Grande Portuguesa" R$ 64,90 obs "sem ervilha"
pedido 1x "Combo Refri 20 unidades" R$ 1.150,00
entrega "Av. Paulista, 1578"
taxa R$ 15,00
cupom FRETEGRATIS
pagamento cartão''',
    "❌ Erro: aspas não fechadas": '''PEDIDO 1x "Açaí 500ml R$ 19,90
PAGAMENTO dinheiro''',
    "❌ Erro: centavos com ponto": '''PEDIDO 2x "Pastel" 8.50''',
    "⚠ Cupom inexistente": '''PEDIDO 1x "Temaki Salmão" R$ 32,00
CUPOM GANHEI100
PAGAMENTO pix'''
}

```

```

seletor_a2 = widgets Dropdown(options=list(exemplos_a2), description="Exemplos:",
                                layout=widgets.Layout(width="60%"))
entrada_a2 = widgets.Textarea(value=exemplos_a2["Pedido completo (delivery)"],
                                layout=widgets.Layout(width="95%", height="190px"))
botao_a2 = widgets.Button(description="🔍 Analisar pedido", button_style="success")
status_a2 = widgets.HTML()
abas_a2 = widgets.Tab(children=[widgets.Output() for _ in range(4)])
for i, nome in enumerate(["🌈 Colorido", "📄 Tokens", "📋 Comanda", "📊 Estatística"]):
    abas_a2.set_title(i, nome)

def ao_escolher_a2(m):
    entrada_a2.value = exemplos_a2[m["new"]]
    ao_clicar_a2(None)

def ao_clicar_a2(_):
    texto = entrada_a2.value
    for aba in abas_a2.children:
        aba.clear_output()
    try:
        tokens = tokenizar_a2(texto)
    except UnexpectedCharacters as erro:
        status_a2.value = erro_lexico_html(texto, erro)
        return
    status_a2.value = f"<b style='color:#2e7d32'>✅ Análise léxica OK – {len(tokens)} tokens reconhecidos.</b>"

    conteudos = [texto_colorido_html(texto, tokens), tabela_tokens_html(tokens),
                  montar_comanda(tokens), estatisticas_html(tokens)]
    for aba, conteudo in zip(abas_a2.children, conteudos):
        with aba:
            display(HTML(conteudo))

seletor_a2.observe(ao_escolher_a2, names="value")
botao_a2.on_click(ao_clicar_a2)
display(widgets.HTML("<h3>🚚 Sistema de Pedidos – Analisador Léxico v2</h3>"),
         seletor_a2, entrada_a2, botao_a2, status_a2, abas_a2)
ao_clicar_a2(None)

```

Roteiro de testes do A2:

Exemplo	O que observar
Pedido completo (delivery)	Total = R\$ 87,46 (subtotal 88,30 – 10% + taxa 7,99)
Pedido grande (evento)	Tudo em minúsculas funciona; R\$ 1.150,00 reconhecido; FRETEGRATIS zera a taxa
✗ Aspas não fechadas	Erro na linha 1, coluna 11, com a dica das aspas
✗ Centavos com ponto	O 8 vira NUMERO e o . gera erro com a dica da vírgula
⚠ Cupom inexistente	Léxico OK, mas a comanda recusa: diferença entre erro léxico e regra de negócio

💡 **Reflexão:** no exemplo "centavos com ponto", o lexer reconheceu 8 como NUMERO antes de falhar. Por que ele não falhou logo no 8?

CENÁRIO B — PIX e Extrato Bancário

Contexto de mercado

O **PIX**, criado pelo Banco Central do Brasil, movimenta bilhões de transações por mês. Toda transferência PIX usa uma **chave**, que pode ser de **4 tipos**:

Tipo de chave	Formato	Exemplo
E-mail	<code>usuario@dominio</code>	<code>maria.souza@gmail.com</code>
CPF / CNPJ	com pontuação	<code>123.456.789-09 /</code> <code>12.345.678/0001-90</code>
Telefone	<code>+55</code> + DDD + número	<code>+5511987654321</code>
Aleatória (EVP)	UUID de 36 caracteres	<code>7d9f3c2a-1b4e-4c8a-9f21-</code> <code>0a6b5e3d7c10</code>

Descobrir **qual é o tipo** de uma chave **só olhando o texto** é, literalmente, um problema de **análise léxica**. Fintechs como Nubank, Inter e PicPay fazem isso ao validar chaves, ler comprovantes e **conciliar extratos** (bater o que entrou e saiu da conta).

A metáfora do caixa do banco 🏦

O lexer é o **caixa** que recebe um envelope cheio de papéis misturados e os separa em pilhas: *"isto é um valor, isto é uma data, isto é um CPF, isto é um e-mail"*. Ele **não** confere se o CPF existe na Receita Federal (isso é semântica). Ele só olha o **formato**.

Exemplo B1 — Leitor de Comprovante PIX (foco: ambiguidade)

Passo 1 — Tabela de tokens

Token	Padrão (resumo)	Exemplo
<code>PIX</code>	palavra reservada	<code>PIX</code>
<code>DIRECAO</code>	<code>enviado</code> ou <code>recebido</code>	<code>ENVIADO</code>
<code>PARA</code> , <code>DE</code> , <code>EM</code>	preposições reservadas	<code>PARA</code>
<code>VALOR</code>	<code>R\$</code> com milhar e centavos	<code>R\$ 1.234,56</code>
<code>DATA</code>	<code>dd/mm/aaaa</code>	<code>10/09/2026</code>
<code>HORA</code>	<code>hh:mm</code>	<code>14:32</code>
<code>CHAVE_EMAIL</code>	<code>algo@dominio.ext</code>	<code>joao@email.com</code>
<code>CHAVE_CPF</code>	<code>000.000.000-00</code>	<code>123.456.789-09</code>
<code>CHAVE_TELEFONE</code>	<code>+55</code> + 10 ou 11 dígitos	<code>+5511987654321</code>
<code>CHAVE_ALEATORIA</code>	UUID (8-4-4-4-12 hexadecimais)	<code>7d9f3c2a-...</code>

Passo 2 — Onde mora a ambiguidade?

Olhe estes lexemas e pense em **quais tokens poderiam começar** em cada um:

Lexema	Candidatos	Quem deve vencer
<code>10/09/2026</code>	<code>DATA</code> ... e um número solto <code>10</code> ?	<code>DATA</code>
<code>14:32</code>	<code>HORA</code> ... ou número <code>14</code> ?	<code>HORA</code>

Lexema	Candidatos	Quem deve vencer
123.456.789-09	CHAVE_CPF	CHAVE_CPF
pix@loja.com.br	PIX (palavra reservada!) ou CHAVE_EMAIL	CHAVE_EMAIL
de@x.com	DE (preposição!) ou CHAVE_EMAIL	CHAVE_EMAIL

O caso do `pix@loja.com.br` é traiçoeiro: o `\b` de `/pix\b/i` **cas**a logo depois de `pix`, porque o `@` não é letra — então é uma fronteira de palavra! Se a palavra reservada vencer, sobra um `@loja.com.br` que nenhum token reconhece.

Passo 3 — A solução: uma escada de prioridades

prioridade 4 → CHAVE_EMAIL (o @ torna o `e-mail` inconfundível)
prioridade 3 → palavras reservadas (PIX, DIRECAO, PARA, DE, `EM`)
prioridade 2 → literais e demais chaves (VALOR, `DATA`, HORA, CPF, TELEFONE, ALEATÓRIA)

Por que é **seguro** colocar o e-mail acima das palavras reservadas? Porque o padrão do e-mail **exige** um `@`. A palavra `PIX` sozinha nunca vai casar com `CHAVE_EMAIL`. Só "roubamos" quando realmente há um e-mail.

Código completo — Exemplo B1 (lexer)

```
# =====  
# EXEMPLO B1 – LEITOR DE COMPROVANTE PIX (uma linha)  
# Cenário: o banco recebe a linha de um comprovante e precisa  
# descobrir QUE TIPO de chave PIX foi usada.  
# Foco da aula: PRIORIDADE e AMBIGUIDADE léxica.  
# =====  
  
gramatica_b1 = r"""  
start: _token*  
_token: PIX | DIRECAO | PARA | DE | EM | VALOR | DATA | HORA  
        | CHAVE_EMAIL | CHAVE_CPF | CHAVE_TELEFONE | CHAVE_ALEATORIA  
  
// ----- Palavras reservadas -----  
PIX.3:    /pix\b/i  
DIRECAO.3: /(enviado|recebido)\b/i  
PARA.3:   /para\b/i  
DE.3:     /de\b/i  
EM.3:     /em\b/i  
  
// ----- Literais -----  
VALOR.2:  /R\$ ?\d{1,3}(\.\d{3})*,\d{2}/           // R$ 1.234,56  
DATA.2:   /\d{2}\\d{2}\\d{4}/                       // 10/09/2026  
HORA.2:   /\d{2}:\d{2}/                             // 14:32  
  
// ----- Os 4 tipos de chave PIX (regras do Banco Central) -----  
CHAVE_EMAIL.4:  /[a-z0-9._+-]+@[a-z0-9-]+(\.[a-z0-9-]+)+/i // maria@gmail.com  
CHAVE_CPF.2:    /\d{3}\.\d{3}\.\d{3}-\d{2}/                // 123.456.789-09  
CHAVE_TELEFONE.2: /\+55\d{10,11}/                          // +5511987654321  
CHAVE_ALEATORIA.2: /[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}/i  
  
%ignore /\t\r\n+/  
"""
```

```
lexer_b1 = Lark(gramatica_b1, parser="lalr", lexer="basic")

def tokenizar_b1(texto):
    return list(lexer_b1.lex(texto))

linha = "PIX ENVIADO R$ 150,00 PARA joao.silva@email.com EM 10/09/2026 14:32"
for t in tokenizar_b1(linha):
    print(f" {t.type:<16} {t.value!r}")
```

Saída esperada:

PIX	'PIX'
DIRECAO	'ENVIADO'
VALOR	'R\$ 150,00'
PARA	'PARA'
CHAVE_EMAIL	'joao.silva@email.com'
EM	'EM'
DATA	'10/09/2026'
HORA	'14:32'

Passo 4 — Laboratório de prioridade (interface)

A interface do B1 tem um **slider** que muda a prioridade da `CHAVE_EMAIL` e **reconstrói o lexer na hora**:

```
gramatica = gramatica_b1.replace("CHAVE_EMAIL.4", f"CHAVE_EMAIL.{prioridade_b1.value}")
lexer = Lark(gramatica, parser="lalr", lexer="basic")
```

Experimento guiado:

- Escolha o exemplo "🔧 E-mail que começa com 'pix'". Com prioridade **4**: tudo certo.
- Arraste o slider para **2**. Resultado: **ERRO** na coluna do `@`. O `pix` virou palavra reservada!
- Volte para **4**. Funciona de novo. 🎉
- Escolha "❌ Telefone sem +55": o lexer não aceita `11987654321`, porque nossa regra exige o formato internacional. Isso é uma **decisão de projeto** do lexer.

```
# =====
# EXEMPLO B1 - INTERFACE + LABORATÓRIO DE PRIORIDADE
# Mude a prioridade da CHAVE_EMAIL e veja o lexer "quebrar"!
# =====

exemplos_b1 = {
    "Chave e-mail": "PIX ENVIADO R$ 150,00 PARA joao.silva@email.com EM 10/09/2026 14:32",
    "Chave CPF": "PIX RECEBIDO R$ 3.200,00 DE 123.456.789-09 EM 05/09/2026 09:15",
    "Chave telefone": "PIX ENVIADO R$ 89,90 PARA +5511987654321 EM 07/09/2026 19:45",
    "Chave aleatória": "PIX ENVIADO R$ 35,00 PARA 7d9f3c2a-1b4e-4c8a-9f21-0a6b5e3d7c10 EM 10/09/2026 14:32",
    "🔧 E-mail que começa com 'pix'": "PIX ENVIADO R$ 10,00 PARA pix@loja.com.br EM 01/09/2026 08:00",
    "❌ Telefone sem +55": "PIX ENVIADO R$ 20,00 PARA 11987654321 EM 02/09/2026 10:10",
}

seletor_b1 = widgets.Dropdown(options=list(exemplos_b1), description="Exemplos:",
                              layout=widgets.Layout(width="60%"))
entrada_b1 = widgets.Text(value=exemplos_b1["Chave e-mail"], layout=widgets.Layout(width="95%"))
prioridade_b1 = widgets.IntSlider(value=4, min=1, max=5, description="Prior. e-mail:",
                                  style={"description_width": "initial"})
```

```

explica_b1 = widgets.HTML("<small>Palavras reservadas têm prioridade <b>3</b>. "
                           "Se o e-mail ficar abaixo de 3, o 'pix' de <code>pix@loja.com.br</code> "
                           "vira palavra reservada e sobra um '@' sem dono.</small>")

botao_b1 = widgets.Button(description="🔍 Ler comprovante", button_style="primary")
saida_b1 = widgets.Output()

def ao_clicar_b1(_):
    saida_b1.clear_output()
    # Recria o lexer com a prioridade escolhida no slider
    gramatica = gramatica_b1.replace("CHAVE_EMAIL.4", f"CHAVE_EMAIL.{prioridade_b1.value}")
    lexer = Lark(gramatica, parser="lalr", lexer="basic")
    texto = entrada_b1.value
    with saida_b1:
        try:
            tokens = list(lexer.lex(texto))
            display(HTML(texto_colorido_html(texto, tokens)))
            chaves = [t for t in tokens if t.type.startswith("CHAVE_")]
            if chaves:
                tipo = chaves[0].type.replace("CHAVE_", "").lower()
                display(HTML(f"<p>🔑 Tipo de chave PIX detectado: <b>{tipo}</b></p>"))
                display(HTML(tabela_tokens_html(tokens)))
            except UnexpectedCharacters as erro:
                erro.dica = "Dica: telefone PIX precisa do +55 (formato internacional)." if erro.char.isdigit() e
lse ""

                display(HTML(erro_lexico_html(texto, erro)))

def ao_escolher_b1(m):
    entrada_b1.value = exemplos_b1[m["new"]]
    ao_clicar_b1(None)

seletor_b1.observe(ao_escolher_b1, names="value")
prioridade_b1.observe(lambda m: ao_clicar_b1(None), names="value")
botao_b1.on_click(ao_clicar_b1)
display(widgets.HTML("<h3>💎 Leitor de Comprovante PIX</h3>"),
         seletor_b1, entrada_b1, widgets.HBox([prioridade_b1, botao_b1]), explica_b1, saida_b1)
ao_clicar_b1(None)

```

🌐 **Mundo real:** este é exatamente o tipo de bug que aparece em produção. Um sistema funciona por meses até que um cliente cadastra o e-mail `pix@empresa.com.br` — e o parser do comprovante quebra. Testes com casos "estranhos" (*edge cases*) salvam empregos.

Exemplo B2 — Conciliação de Extrato Bancário (versão profissional)

O que muda do B1 para o B2

Recurso	B1	B2
Entrada	uma linha	extrato do mês inteiro
Operações	só PIX	PIX, TED, BOLETO, TARIFA, SALDO INICIAL
Chaves	4 tipos	+ CNPJ (empresas)
Token de 2 palavras	não	<code>SALDO INICIAL</code> vira um token

Recurso	B1	B2
Privacidade	não	maskamento LGPD
Saída	tabela	extrato conciliado + painel + alertas

Passo 1 — Um token com duas palavras

```
SALDO_INICIAL.3: /saldo\s+inicial\b/i
```

O `\s+` aceita um ou mais espaços **no meio** do token. Muitas linguagens fazem isso: em SQL, `ORDER BY` e `GROUP BY` costumam ser tratados como uma unidade.

Passo 2 — CNPJ x CPF

```
CHAVE_CNPJ.2: /\d{2}\.\d{3}\.\d{3}\\d{4}-\d{2}/ // 12.345.678/0001-90
CHAVE_CPF.2: /\d{3}\.\d{3}\.\d{3}-\d{2}/ // 123.456.789-09
```

Eles **não** entram em conflito: o CNPJ começa com **2** dígitos e tem uma `/`; o CPF começa com **3** dígitos. Os padrões são **mutuamente exclusivos** — a melhor situação possível para um projetista de lexer.

Passo 3 — LGPD: mascarando lexemas

A **Lei Geral de Proteção de Dados** exige cuidado ao exibir dados pessoais. Como o lexer já sabe **o tipo** de cada token, mascarar fica trivial:

Token	Lexema original	Exibido
CHAVE_CPF	123.456.789-09	***.456.789-**
CHAVE_EMAIL	maria.souza@gmail.com	m***@gmail.com
CHAVE_TELEFONE	+5511987654321	+5511*****4321
CHAVE_ALEATORIA	7d9f3c2a-1b4e-...	7d9f3c2a-****
CHAVE_CNPJ	12.345.678/0001-90	(sem máscara — dado de empresa)

 **DevSecOps:** ferramentas como **GitHub Secret Scanning**, **Gitleaks** e **TruffleHog** usam exatamente esta técnica — padrões regex por *tipo de segredo* (chave AWS, token do GitHub, CPF...) — para impedir que dados sensíveis vazem em commits.

Código completo — Exemplo B2 (lexer + LGPD)

```
# =====
# EXEMPLO B2 - CONCILIAÇÃO DE EXTRATO BANCÁRIO (multilinha)
# Cenário: fintech importa o extrato do mês e precisa
#   (1) reconhecer cada token, (2) mascarar dados pessoais (LGPD),
#   (3) calcular entradas, saídas e saldo final.
# =====

gramatica_b2 = r"""
start: _token*
_token: SALDO_INICIAL | PIX | TED | BOLETO | TARIFA | DIRECAO | PARA | DE | EM
      | VALOR | DATA | HORA | DESCRICAO
      | CHAVE_EMAIL | CHAVE_CPF | CHAVE_CNPJ | CHAVE_TELEFONE | CHAVE_ALEATORIA

// ----- Palavras reservadas (prioridade 3) -----
SALDO_INICIAL.3: /saldo\s+inicial\b/i // duas palavras = UM token só!
PIX.3: /pix\b/i
```

```

TED.3:      /ted\b/i
BOLETO.3:   /boleto\b/i
TARIFA.3:   /tarifa\b/i
DIRECAO.3:  /(enviado|recebido|pago)\b/i
PARA.3:     /para\b/i
DE.3:       /de\b/i
EM.3:       /em\b/i

// ----- Literais -----
VALOR.2:    /R\$ ?\d{1,3}(\.\d{3})*,\d{2}/
DATA.2:     /\d{2}\\d{2}\\d{4}/
HORA.2:     /\d{2}:\d{2}/
DESCRICAO:  /"[^"\n]*/

// ----- Chaves PIX -----
CHAVE_EMAIL.4:  /[a-z0-9._+-]+@[a-z0-9-]+\.[a-z0-9-]+\b/i
CHAVE_CNPJ.2:   /\d{2}\.\d{3}\.\d{3}\\d{4}-\d{2}/           // 12.345.678/0001-90
CHAVE_CPF.2:    /\d{3}\.\d{3}\.\d{3}-\d{2}/
CHAVE_TELEFONE.2: /\+55\d{10,11}/
CHAVE_ALEATORIA.2: /[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}/i

// ----- Ruído -----
COMENTARIO: /#[^"\n]*/
%ignore COMENTARIO
%ignore /\t\r\n+/
"""

lexer_b2 = Lark(grammar_b2, parser="lalr", lexer="basic")

def tokenizar_b2(texto):
    try:
        return list(lexer_b2.lex(texto))
    except UnexpectedCharacters as erro:
        dicas = {
            "'": "Descrição com aspas abertas e não fechadas.",
            ".": "Valores usam vírgula nos centavos (R$ 10,50).",
            "@": "E-mail incompleto? Formato esperado: nome@dominio.com"
        }
        erro.dica = dicas.get(erro.char, "Símbolo fora do alfabeto do extrato. "
                                "Telefone precisa de +55; CPF precisa de pontos e traço.")
        raise

# ----- LGPD: mascaramento de dados pessoais -----
def mascarar(token):
    """Esconde parte do lexema de chaves que identificam PESSOAS."""
    v = token.value
    if token.type == "CHAVE_CPF":          # 123.456.789-09 -> ***.456.789-**
        return "****" + v[3:12] + "****"
    if token.type == "CHAVE_EMAIL":       # maria.souza@gmail.com -> m***@gmail.com
        usuario, dominio = v.split("@")
        return usuario[0] + "****@" + dominio
    if token.type == "CHAVE_TELEFONE":    # +5511987654321 -> +5511*****4321
        return v[:5] + "*" * (len(v) - 9) + v[-4:]
    if token.type == "CHAVE_ALEATORIA":   # mostra só o começo
        return v[:8] + "-****"

```

```

return v # CNPJ é de empresa: pode aparecer

def valor_em_reais(lexema):
    return float(lexema.replace("R$", "").strip().replace(".", "").replace(",", "."))

# Teste no console
teste = 'PIX ENVIADO R$ 1.200,00 PARA maria.souza@gmail.com EM 06/09/2026 10:02 "Aluguel"'
for t in tokenizar_b2(teste):
    print(f" {t.type:<16} {t.value!r:<40} -> exibido como {mascarar(t)!r}")

```

Saída esperada:

```

PIX          'PIX'          -> exibido como 'PIX'
DIRECAO      'ENVIADO'       -> exibido como 'ENVIADO'
VALOR        'R$ 1.200,00'    -> exibido como 'R$ 1.200,00'
PARA         'PARA'          -> exibido como 'PARA'
CHAVE_EMAIL  'maria.souza@gmail.com' -> exibido como 'm***@gmail.com'
EM           'EM'            -> exibido como 'EM'
DATA         '06/09/2026'     -> exibido como '06/09/2026'
HORA         '10:02'          -> exibido como '10:02'
DESCRICA0    '"Aluguel"'      -> exibido como '"Aluguel"'

```

Passo 4 — Agrupando tokens por linha

Como ignoramos as quebras de linha, como saber onde termina uma transação? Usando o atributo `t.line` que o Lark guarda em cada token:

```

linhas = {}
for t in tokens:
    linhas.setdefault(t.line, []).append(t)

```

Cada linha do extrato vira uma lista de tokens — e cada lista vira uma transação.

Passo 5 — Regra de negócio real: limite do PIX noturno

Por regulamentação do Banco Central, o PIX de pessoa física no **período noturno (20h às 6h)** tem limite padrão de **R\$ 1.000,00** (o cliente pode alterar pelo app). A função `alertas` usa o token `HORA` para sinalizar transações acima do limite. Mais uma vez: **o lexer entregou as peças; a regra de negócio é outra camada.**

Código completo — Exemplo B2 (interface)

```

# =====
# EXEMPLO B2 - INTERFACE: EXTRATO CONCILIADO + LGPD + ALERTAS
# =====

from collections import Counter

def brl(v):
    """Formata número no padrão brasileiro: -1200.5 -> '- R$ 1.200,50'."""
    txt = f"R$ {abs(v):.2f}".replace(",", "X").replace(".", ",").replace("X", ".")
    return ("- " if v < 0 else "") + txt

def conciliar(tokens):
    """Agrupar os tokens POR LINHA (t.line) e transforma cada linha em uma transação.
    De novo: isto é uma espiada no parser - o lexer só entregou as peças."""
    linhas = {}

```

```

for t in tokens:
    linhas.setdefault(t.line, []).append(t)

saldo_inicial, transacoes = 0.0, []
for num, toks in sorted(linhas.items()):
    tipos = [t.type for t in toks]
    pega = lambda tipo: next((t for t in toks if t.type == tipo), None)
    valor = pega("VALOR")
    if tipos[0] == "SALDO_INICIAL" and valor:
        saldo_inicial = valor_em_reais(valor.value)
        continue
    if tipos[0] not in ("PIX", "TED", "BOLETO", "TARIFA") or not valor:
        continue
    direcao = pega("DIRECAO")
    entrada = direcao is not None and direcao.value.upper() == "RECEBIDO"
    chave = next((t for t in toks if t.type.startswith("CHAVE_")), None)
    hora = pega("HORA")
    transacoes.append({
        "linha": num, "tipo": toks[0].value.upper(),
        "direcao": direcao.value.upper() if direcao else "DÉBITO",
        "valor": valor_em_reais(valor.value) * (1 if entrada else -1),
        "chave": chave, "data": pega("DATA").value if pega("DATA") else "-",
        "hora": hora.value if hora else "-",
        "descricao": pega("DESCRICAO").value.strip(' ') if pega("DESCRICAO") else ""
    })
return saldo_inicial, transacoes

def alertas(transacoes):
    """Regras de negócio inspiradas no mundo real (Banco Central):
    PIX noturno (20h-6h) tem limite padrão de R$ 1.000,00 para pessoa física."""
    avisos = []
    for tr in transacoes:
        if tr["tipo"] == "PIX" and tr["valor"] < 0 and tr["hora"] != "-":
            h = int(tr["hora"][:2])
            if (h >= 20 or h < 6) and abs(tr["valor"]) > 1000:
                avisos.append(f"🌙 Linha {tr['linha']}: PIX noturno de {brl(abs(tr['valor']))} acima do limite
de R$ 1.000,00")
    return avisos

def extrato_html(tokens, lgpd):
    saldo_ini, trs = conciliar(tokens)
    saldo = saldo_ini
    linhas = f"<tr><td colspan=5><b>Saldo inicial</b></td><td align=right><b>{brl(saldo)}</b></td></tr>"
    for tr in trs:
        saldo += tr["valor"]
        cor = "#2e7d32" if tr["valor"] > 0 else "#c62828"
        chave = tr["chave"]
        chave_txt = "-" if not chave else (mascarar(chave) if lgpd else chave.value)
        tipo_chave = "" if not chave else f"<br><small>{chave.type.replace('CHAVE_', '').lower()}</small>"
        linhas += (f"<tr><td>{tr['data']} {tr['hora']}</td><td>{tr['tipo']} {tr['direcao']}</td>"
            f"<td><code>{html.escape(chave_txt)}</code>{tipo_chave}</td>"
            f"<td>{html.escape(tr['descricao'])}</td>"
            f"<td align=right style='color:{cor}'>{brl(tr['valor'])}</td>"

```

```

        f"<td align=right>{brl(saldo)}</td></tr>")
    return f"""<table style='border-collapse:collapse;font-size:13px' border=1 cellpadding=5>
        <tr style='background:#263238;color:white'><th>Data/Hora</th><th>Operação</th>
        <th>Contraparte</th><th>Descrição</th><th>Valor</th><th>Saldo</th></tr>{linhas}</table>"""

def painel_html(tokens):
    saldo_ini, trs = conciliar(tokens)
    entradas = sum(t["valor"] for t in trs if t["valor"] > 0)
    saidas = -sum(t["valor"] for t in trs if t["valor"] < 0)
    chaves = Counter(t.type.replace("CHAVE_", "").lower() for t in tokens if t.type.startswith("CHAVE_"))
    cartao = lambda titulo, valor, cor: (f"<div style='display:inline-block;margin:4px;padding:10px 16px;"
        f"border-radius:8px;background:{cor};color:white'"
        f"<small>{titulo}</small><br><b style='font-size:18px'>{valor}</b></div>")

    avisos = alertas(trs)
    return (cartao("Entradas", brl(entradas), "#2e7d32") + cartao("Saídas", brl(saidas), "#c62828")
        + cartao("Saldo final", brl(saldo_ini + entradas - saidas), "#1565c0")
        + cartao("Transações", len(trs), "#455a64")
        + "<h4>🔑 Tipos de chave PIX encontrados</h4>"
        + ".join(f"<code>{k}</code>: {n}<br>" for k, n in chaves.items())
        + "<h4>🚨 Alertas</h4>" + ("<br>".join(avisos) or "Nenhum alerta."))

extrato_exemplo = '''# Extrato – Conta Corrente | Setembro/2026
SALDO INICIAL R$ 2.500,00
PIX RECEBIDO R$ 3.200,00 DE 12.345.678/0001-90 EM 05/09/2026 09:15 "Salário"
PIX ENVIADO R$ 1.200,00 PARA maria.souza@gmail.com EM 06/09/2026 10:02 "Aluguel"
PIX ENVIADO R$ 89,90 PARA +5511987654321 EM 07/09/2026 19:45 "Pizza de sexta"
BOLETO PAGO R$ 149,90 EM 08/09/2026 08:30 "Internet fibra"
PIX RECEBIDO R$ 50,00 DE 123.456.789-09 EM 09/09/2026 12:10 "Racha do almoço"
PIX ENVIADO R$ 35,00 PARA 7d9f3c2a-1b4e-4c8a-9f21-0a6b5e3d7c10 EM 10/09/2026 14:32 "Doação ONG"
TED ENVIADO R$ 500,00 PARA 98.765.432/0001-10 EM 10/09/2026 16:00 "Curso de inglês"
TARIFA R$ 12,50 EM 10/09/2026 23:59 "Pacote de serviços"'''

exemplos_b2 = {
    "Extrato do mês": extrato_exemplo,
    "🌙 Com PIX noturno alto": '''SALDO INICIAL R$ 5.000,00
PIX ENVIADO R$ 1.500,00 PARA +5521999998888 EM 11/09/2026 22:40 "Conserto do carro"
PIX ENVIADO R$ 300,00 PARA ana@empresa.com.br EM 11/09/2026 23:10 "Presente"''',
    "❌ Erro: CPF sem pontuação": '''SALDO INICIAL R$ 100,00
PIX RECEBIDO R$ 20,00 DE 12345678909 EM 01/09/2026 10:00''',
    "❌ Erro: descrição sem fechar aspas": '''PIX ENVIADO R$ 15,00 PARA a@b.com EM 01/09/2026 10:00 "Café"''',
}

seletor_b2 = widgets.Dropdown(options=list(exemplos_b2), description="Exemplos:",
    layout=widgets.Layout(width="60%"))
entrada_b2 = widgets.Textarea(value=extrato_exemplo, layout=widgets.Layout(width="98%", height="220px"))
lgpd_b2 = widgets.Checkbox(value=True, description="🔒 Mascaram dados pessoais (LGPD)")
botao_b2 = widgets.Button(description="🏠 Conciliar extrato", button_style="info")
status_b2 = widgets.HTML()
abas_b2 = widgets.Tab(children=[widgets.Output() for _ in range(4)])
for i, nome in enumerate(["🌈 Colorido", "📄 Tokens", "📄 Extrato", "📄 Painel"]):
    abas_b2.set_title(i, nome)

```

```
def ao_clicar_b2(_):
    texto = entrada_b2.value
    for aba in abas_b2.children:
        aba.clear_output()
    try:
        tokens = tokenizar_b2(texto)
    except UnexpectedCharacters as erro:
        status_b2.value = erro_lexico_html(texto, erro)
    return

status_b2.value = f"<b style='color:#2e7d32'>✅ {len(tokens)} tokens reconhecidos.</b>"
lgpd = lgpd_b2.value
tokens_exibidos = [t.update(value=mascarar(t)) if lgpd else t for t in tokens]
conteudos = [texto_colorido_html(texto, tokens, mascarar if lgpd else None),
               tabela_tokens_html(tokens_exibidos),
               extrato_html(tokens, lgpd), painel_html(tokens)]
for aba, c in zip(abas_b2.children, conteudos):
    with aba:
        display(HTML(c))

def ao_escolher_b2(m):
    entrada_b2.value = exemplos_b2[m["new"]]
    ao_clicar_b2(None)

seletor_b2.observe(ao_escolher_b2, names="value")
lgpd_b2.observe(lambda m: ao_clicar_b2(None), names="value")
botao_b2.on_click(ao_clicar_b2)
display(widgets.HTML("<h3>🏠 Conciliação de Extrato – Analisador Léxico PIX v2</h3>"),
        seletor_b2, entrada_b2, widgets.HBox([botao_b2, lgpd_b2]), status_b2, abas_b2)
ao_clicar_b2(None)
```

Roteiro de testes do B2:

Exemplo	O que observar
Extrato do mês	Entradas R\$ 3.250,00 · Saídas R\$ 1.987,30 · Saldo final R\$ 3.762,70 · 6 chaves de 5 tipos
Desligar o LGPD	Os dados completos aparecem nas abas Colorido, Tokens e Extrato
🌙 PIX noturno alto	Alerta para o PIX de R\$ 1.500,00 às 22:40
❌ CPF sem pontuação	<code>12345678909</code> não casa com nenhum token → erro na linha 2, coluna 26
❌ Descrição sem aspas	Erro apontando o <code>"</code> que não foi fechado

5. Conexões com o Mercado (2026)

Onde	Como a análise léxica aparece
VS Code / Cursor / GitHub Copilot	O realce de sintaxe e o autocompletar começam por tokenização (TextMate grammars e Tree-sitter)
LLMs (GPT, Claude, Llama)	O primeiro passo de todo modelo de linguagem é um tokenizador (BPE) — outro tipo de "lexer", estatístico

Onde	Como a análise léxica aparece
Bancos e fintechs	Leitura de arquivos CNAB, extratos OFX, validação de chaves PIX e códigos de barras de boleto
Apps de delivery	Integração com PDVs de restaurantes, impressão de comandas, leitura de cardápios
DevSecOps	Gitleaks, TruffleHog e GitHub Secret Scanning detectam segredos por padrões léxicos
Compiladores de verdade	Clang/LLVM, o compilador do Rust, o V8 (JavaScript) e o TypeScript Compiler API têm lexers escritos à mão, altamente otimizados
Ferramentas geradoras	Lex/Flex , ANTLR4 , Lark , Pest (Rust) e Chevrotain (JS) geram lexers a partir de especificações — como fizemos hoje

💬 **Discussão em duplas:** o tokenizador de um LLM e o lexer de um compilador fazem a "mesma coisa"? Liste **uma semelhança e uma diferença**. (Pista: um usa regras escritas por humanos; o outro aprende as "palavras" a partir de estatística.)

6. Exercícios

📄 Parte 1 — No caderno (sem computador)

Exercício 1 — Tokenização à mão (PedidoLang, gramática do A2). Monte a tabela

TOKEN	LEXEMA	LINHA	COLUNA
-------	--------	-------	--------

 para a entrada abaixo:

```
# almoço
pedido 3X "Pastel de Queijo" R$ 8,50
PAGAMENTO Cartão
```

Exercício 2 — Detetive de prioridades. Para cada lexema, diga qual token o lexer do **B1** produz e **justifique pela prioridade**:

a) `em@banco.com` b) `EMPRESA` c) `+5511987654321` d) `09:45`

Exercício 3 — Caça ao erro léxico. Indique **linha, coluna e caractere** do primeiro erro léxico (gramática do B2):

```
SALDO INICIAL R$ 300,00
PIX ENVIADO R$ 45,00 PARA joao@email.com EM 01/09/2026 10:00
```

Exercício 4 — Escreva a regex. Escreva padrões Lark para:

- a) Código de rastreio dos Correios (2 letras, 9 dígitos, 2 letras — ex.: `BR123456789BR`)
- b) Placa de carro Mercosul (ex.: `ABC1D23`)
- c) CEP (ex.: `01310-100`)

Exercício 5 — Reflexão. No Exemplo A2, a entrada `CUPOM GANHEI100` passa pelo lexer sem erro, mas a comanda recusa o cupom. Em qual fase do compilador essa verificação deveria acontecer? Por que o lexer **não deve** fazê-la?

💻 Parte 2 — No Colab / VS Code

Nível guiado 🟢

1. **A1:** adicione o token `OBS` (literal `"OBS"`) e teste `PEDIDO 1x "Pizza" R$ 50,00 OBS "sem cebola"` .
2. **A2:** adicione a forma de pagamento `vr` (vale-refeição abreviado) e o cupom `DESC20` (20%).
3. **B1:** adicione o token `CHAVE_CNPJ` ao lexer do B1 (copie do B2) e teste com `12.345.678/0001-90` .
4. **B2:** adicione a operação `SAQUE` (sempre saída) e teste com `SAQUE R$ 200,00 EM 05/09/2026 18:00` .

Nível desafio

- 1. **A2:** aceite também preços sem R\$, como 25,90 . Cuidado: esse padrão entra em conflito com NUMERO ? Com QTD ? Documente a prioridade escolhida.
- 2. **B2:** aceite telefone sem o +55 (11987654321) e explique o novo conflito que surge com números de 11 dígitos. Qual informação extra ajudaria a desambiguar?
- 3. **B2:** crie um novo alerta: "PIX para chave aleatória acima de R\$ 500,00" (padrão comum em golpes).

7. 🏆 Desafio para Entrega — "Meu Analisador Léxico de Mercado"

A missão

Você foi contratado(a) por uma empresa para criar o **analisador léxico** de uma mini-linguagem do mundo real. Escolha **um** dos temas abaixo (ou proponha outro, com aprovação do professor):

#	Tema	Exemplo de entrada
1	🚗 Corrida por aplicativo (Uber/99)	CORRIDA DE "Av. Paulista, 1000" PARA "Aeroporto GRU" CATEGORIA comfort AGENDAR 12/09/2026 06:30 PAGAMENTO pix
2	📦 Rastreo de encomendas (Correios/Mercado Livre)	RASTREIO BR123456789BR STATUS "saiu para entrega" CEP 01310-100 EM 10/09/2026 08:15
3	🧾 Cupom fiscal eletrônico (NFC-e)	ITEM 001 "Arroz 5kg" QTD 2 UN VL R\$ 27,90 CNPJ 12.345.678/0001-90
4	🎫 Ingressos de eventos (Sympla/Ticketmaster)	INGRESSO 2x "Festival de Inverno" SETOR pista LOTE 2 MEIA R\$ 180,00 DATA 20/07/2026
5	🎧 Comandos de playlist (Spotify)	PLAYLIST "Treino" ADICIONAR "Nome da faixa" DURACAO 03:45 GENERO rock ORDENAR popularidade

Requisitos obrigatórios

- 1. **Tabela de tokens** documentada (token, descrição, regex, exemplo, prioridade).
- 2. No mínimo **12 tipos de token**.
- 3. No mínimo **4 palavras reservadas**, insensíveis a maiúsculas (/ i) e com fronteira de palavra (\b).
- 4. No mínimo **3 literais** definidos por expressão regular (ex.: valor, data, código especial do domínio).
- 5. Pelo menos **1 conflito de prioridade** identificado, resolvido e **explicado em comentário**.
- 6. **Comentários** (#) e espaços ignorados.
- 7. **Erros léxicos** com linha, coluna e **pelo menos 2 dicas amigáveis** específicas do domínio.
- 8. **Interface** com ipywidgets (ou biblioteca similar: Gradio, Streamlit, Panel...) com: área de entrada, botão, texto colorido e tabela de tokens.
- 9. **Casos de teste:** pelo menos **3 entradas válidas** e **2 inválidas**.
- 10. **Diário de ambiguidade:** um parágrafo explicando um conflito que você encontrou e como resolveu (como o caso pix@loja.com.br).

Bônus (até +1,0 ponto)

- Uma aba de **pós-processamento** (resumo, totais, alertas), como a comanda do A2 ou o painel do B2.

- Conversão de lexemas em **valores** (texto → número, texto → data).
- **Mascaramento** de dados sensíveis (LGPD).

Critérios de avaliação

Critério	Peso
Tabela de tokens e expressões regulares corretas	2,5
Tratamento de prioridade e ambiguidade (com explicação)	2,0
Tratamento de erros léxicos (linha, coluna, dicas)	2,0
Interface funcional e clara	1,5
Casos de teste (válidos e inválidos)	1,0
Diário de ambiguidade e organização do código	1,0
Total	10,0

Modelo inicial (já está no notebook)

A última célula do notebook traz um **modelo com** `TODO` s e uma função genérica `interface_lexer(titulo, tokenizar, casos)` que monta a interface para **qualquer** lexer seu:

```
# =====
# 🏆 DESAFIO – MODELO INICIAL (copie esta célula e complete os TODOs)
# =====
# Tema escolhido: _____ (ex.: Corrida de App, Rastreo de Encomenda...)
# Integrantes: _____
# -----

gramatica_desafio = r"""
start: _token*
_token: INICIO | EXEMPLO_TEXTO | EXEMPLO_NUMERO    // TODO: liste TODOS os seus tokens aqui

// ----- 1) Palavras reservadas (mín. 4, com /i e \b) -----
INICIO.3: /inicio\b/i          // TODO: troque pelas palavras do seu domínio

// ----- 2) Literais (mín. 3 tipos com regex) -----
EXEMPLO_TEXTO: /"[^"\n]*"/
EXEMPLO_NUMERO: /\d+/

// ----- 3) Tokens especiais do domínio -----
// TODO: ex.: código de rastreo /[A-Z]{2}\d{9}[A-Z]{2}/, placa /[A-Z]{3}\d[A-Z0-9]\d{2}/ ...

// ----- 4) Ruído -----
COMENTARIO: /#[^\n]*/
%ignore COMENTARIO
%ignore /[ \t\r\n]+/
"""

lexer_desafio = Lark(grammar_desafio, parser="lalr", lexer="basic")

DICAS_DESAFIO = {
    '': "Aspas abertas e não fechadas.",    # TODO: crie pelo menos 2 dicas do SEU domínio
}
```

```

def tokenizar_desafio(texto):
    try:
        return list(lexer_desafio.lex(texto))
    except UnexpectedCharacters as erro:
        erro.dica = DICAS_DESAFIO.get(erro.char, "Caractere fora do alfabeto da linguagem.")
        raise

# ----- Casos de teste (mín. 3 válidos e 2 inválidos) -----
casos_desafio = {
    "✅ Válido 1": 'INICIO "teste" 42',
    "❌ Inválido 1": 'INICIO "sem fechar',
    # TODO: adicione os seus
}

def testar(tokenizar, casos):
    """Roda todos os casos e mostra um relatório rápido no console."""
    for nome, texto in casos.items():
        try:
            toks = tokenizar(texto)
            print(f"{nome:<22} -> {len(toks):>3} tokens: {[t.type for t in toks][:8]}...")
        except UnexpectedCharacters as e:
            print(f"{nome:<22} -> ERRO na linha {e.line}, coluna {e.column} ({e.char!r})")

def interface_lexer(titulo, tokenizar, casos):
    """Interface genérica reutilizável: serve para QUALQUER lexer seu."""
    seletor = widgets.Dropdown(options=list(casos), description="Casos:")
    entrada = widgets.Textarea(value=list(casos.values())[0],
                                layout=widgets.Layout(width="95%", height="150px"))
    botao = widgets.Button(description="🔍 Analisar", button_style="primary")
    saida = widgets.Output()

    def analisar(_):
        saida.clear_output()
        with saida:
            try:
                toks = tokenizar(entrada.value)
                display(HTML(texto_colorido_html(entrada.value, toks)))
                display(HTML(tabela_tokens_html(toks)))
            except UnexpectedCharacters as e:
                display(HTML(erro_lexico_html(entrada.value, e)))

    def escolher(m):
        entrada.value = casos[m["new"]]
        analisar(None)

    seletor.observe(escolher, names="value")
    botao.on_click(analisar)
    display(widgets.HTML(f"<h3>{titulo}</h3>"), seletor, entrada, botao, saida)
    analisar(None)

testar(tokenizar_desafio, casos_desafio)
interface_lexer("🤖 Meu Analisador Léxico", tokenizar_desafio, casos_desafio)
# TODO (bônus): adicione abas com um resumo/pós-processamento, como no A2 e no B2.

```

📁 **Formato de entrega:** será explicado em aula.

8. Checklist de Autoavaliação

Antes de sair da aula, marque o que você já consegue fazer:

- ☐ Explicar a diferença entre **token**, **lexema** e **padrão**
- ☐ Escrever um token no Lark com **string literal** e com **regex**
- ☐ Usar `%ignore` para descartar espaços e comentários
- ☐ Usar **prioridade** (`TOKEN. 2`) para resolver um conflito
- ☐ Explicar por que `\b` e `/i` são úteis em palavras reservadas
- ☐ Capturar `UnexpectedCharacters` e mostrar **linha e coluna**
- ☐ Usar `t.line`, `t.column`, `t.start_pos` e `t.end_pos`
- ☐ Criar uma interface com `Text / Textarea`, `Button`, `Output` e `on_click`
- ☐ Diferenciar um **erro léxico** de uma **regra de negócio**

9. Referências

- CARMO, Robson. **Compiladores**. São Paulo: FMU, 2026. (Unidade 5 — Análise Léxica; Unidade 6 — Autômatos.)
- AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson, 2008. (Cap. 3 — Análise Léxica.)
- MENEZES, Paulo Blauth. **Linguagens Formais e Autômatos**. 6. ed. Porto Alegre: Bookman, 2011.
- COOPER, Keith; TORCZON, Linda. **Construindo Compiladores**. 2. ed. Rio de Janeiro: Elsevier, 2014.
- LARK. **Lark — a parsing toolkit for Python** (documentação oficial). Disponível em: <https://lark-parser.readthedocs.io>
- JUPYTER. **ipywidgets** (documentação oficial). Disponível em: <https://ipywidgets.readthedocs.io>
- BANCO CENTRAL DO BRASIL. **Pix — Regulamento e perguntas frequentes**. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/pix>
- BRASIL. **Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)**.

Apêndice — Gabarito da Parte 1 (para o professor)

Exercício 1

TOKEN	LEXEMA	LINHA	COLUNA
PEDIDO	<code>pedido</code>	2	1
QTD	<code>3X</code>	2	8
TEXTO	<code>"Pastel de Queijo"</code>	2	11
PRECO	<code>R\$ 8,50</code>	2	30
PAGAMENTO	<code>PAGAMENTO</code>	3	1
FORMA_PGTO	<code>Cartão</code>	3	11

O comentário `# almoço` não gera token, mas ocupa a linha 1.

Exercício 2

- a) `CHAVE_EMAIL` — prioridade 4 vence a palavra reservada `EM` (3).

- b) **Erro léxico** na coluna 1 (E). A palavra reservada EM não casa, pois `/em\b/i` exige fronteira de palavra e depois de EM vem P . Como o B1 não tem token de identificador, nenhum padrão reconhece a palavra.
- c) CHAVE_TELEFONE .
- d) HORA .

Exercício 3 — Linha 2, coluna 27, caractere j . Por quê? O padrão CHAVE_EMAIL exige letra ou dígito logo depois do @ ; em joao@@mail.com vem um segundo @ , então o e-mail **não casa por inteiro**. Como nenhum outro token do B2 começa com a letra j , o lexer acusa o erro no **início** do lexema, e não no @ . (Ótimo gancho para discutir: o erro nem sempre aparece onde o humano “vê” o problema.)

Exercício 4

- a) RASTREIO_COD: `/[A-Z]{2}\d{9}[A-Z]{2}/`
- b) PLACA: `/[A-Z]{3}\d[A-Z]\d{2}/`
- c) CEP: `/\d{5}-\d{3}/`

Exercício 5 — Análise **semântica** (regra de negócio). O lexer só verifica o **formato** (o cupom é uma sequência válida de letras e dígitos); saber se ele **existe** exige consultar dados (tabela de cupons), o que está fora do papel da análise léxica.

 Obrigado!

Lembre-se:

"A melhor forma de aprender é fazendo!"

Nos vemos na próxima aula! 🙌

Professor: Hercules Ramos

Email: hercules.ramos@fmu.br

 [LinkedIn](#)

Copyright © 2026 Prof. Hercules Ramos

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).